

ANALISI OOP

Il Gioco di Jeopardy!

Esercizi di analisi orientata agli oggetti in C++

Come usare questo documento

Questo documento ti guida nell'analisi del gioco di Jeopardy! usando i concetti della Programmazione Orientata agli Oggetti (OOP) che hai studiato. Non si tratta di scrivere il codice completo del gioco, ma di allenare il tuo pensiero da progettista: capire quali classi servono, cosa devono sapere e cosa devono fare.

Per ogni sezione troverai:

- Una breve descrizione della competenza OOP affrontata
- Il modello delle classi di Jeopardy! come punto di riferimento
- Domande aperte da rispondere con ragionamenti motivati
- Una traccia (hint) per orientarti se sei in difficoltà

⚠ **Attenzione alle domande trabocchetto** (segnalate con il simbolo ⚠): sono domande con una risposta intuitiva ma errata. Scopri perché!

Jeopardy! in poche parole

Leggi attentamente: da questa descrizione nascono le classi

Concetto di base

Il meccanismo centrale del gioco è semplice ma originale: ogni "risposta" viene presentata sotto forma di affermazione (es. *"È la capitale della Francia"*), e il concorrente deve rispondere obbligatoriamente con una domanda (es. *"Cos'è Parigi?"*). Non formulare la risposta come domanda equivale a perdere il turno.

Il tabellone

Il tabellone di gioco è una griglia composta da:

- **6 categorie** tematiche (storia, scienza, letteratura, cultura pop, ecc.)
- **5 livelli di difficoltà** per ogni categoria, con valori in denaro crescenti

I concorrenti scelgono una casella alla volta (es. *"Capitali europee per 400"*), il conduttore legge l'indizio, e chi suona il campanello per primo risponde.

Svolgimento della partita

La partita si divide in **tre round**:

1. Jeopardy! Round Il tabellone ha valori che vanno da 200 a 1.000 dollari. Nascosta in una o due caselle c'è una **Daily Double**: chi la trova può scommettere fino a tutti i propri soldi su un singolo indizio.

2. Double Jeopardy! Round Il tabellone si raddoppia (da 400 a 2.000 dollari) e ci sono **due Daily Double**. Le poste si alzano e le rimonte diventano possibili.

3. Final Jeopardy! Viene annunciata una sola categoria. Tutti i concorrenti **scrivono la loro scommessa** in segreto (fino al totale accumulato), poi vedono l'indizio e scrivono la risposta. Chi alla fine ha il punteggio più alto vince e torna il giorno successivo.

Regole chiave

- Risposta **errata** → si perde il valore della casella
- Risposta **corretta** → si guadagna il valore
- Si può andare **in negativo** se si sbaglia molto
- Il vincitore **mantiene i soldi** e torna a giocare; gli altri ricevono premi consolatori fissi

Struttura del Tabellone

Livello	Valore Punti
1 (facile)	200
2	400
3	600
4	800
5 (difficile)	1000

Professional Sports	Celebrities	Movies & TV	Geography	Human Body	2016
\$200	\$200	\$200	\$200	\$200	\$200
\$400	\$400	\$400	\$400	\$400	\$400
\$600	\$600	\$600	\$600	\$600	\$600
\$800	\$800	\$800	\$800	\$800	\$800
\$1000	\$1000	\$1000	\$1000	\$1000	\$1000

Azioni del Concorrente

- Scegli Cella — il Concorrente seleziona categoria e livello dal Tabellone.
- Risposta (Buzzer) — il Concorrente suona il buzzer e fornisce la «domanda» corretta.
- Daily Double — cella speciale: il Concorrente può scommettere parte del suo punteggio.

Glossario Minimo

Termine	Significato
Cella	Una voce del tabellone con categoria, livello, risposta e domanda corretta.
Tabellone	La griglia di celle organizzate per categoria e livello di difficoltà.
Buzzer	Il dispositivo con cui il Concorrente segnala di voler rispondere.
Daily Double	Cella speciale che consente al Concorrente di puntare punti extra.
Final Jeopardy	Round finale in cui tutti i concorrenti scommettono parte del punteggio.
Conduttore	Il conduttore che presenta le risposte e valida le domande dei concorrenti.

Le classi di Jeopardy! — il modello di riferimento

Queste sono le classi che useremo come riferimento per tutti gli esercizi. Studialo bene prima di rispondere alle domande.

Responsabilità delle Classi

Ogni classe ha una responsabilità precisa. La tabella mostra cosa contiene e cosa sa fare ciascuna.

Classe	Contiene	Sa fare
Cella	Categoria (stringa) Livello (1–5) Testo risposta Domanda corretta Stato: disponibile / già usata	Dire il proprio valore in punti Passare da disponibile a usata Verificare se una risposta del concorrente è corretta

Classe	Contiene	Sa fare
Tabellone	Matrice di oggetti Cella (categorie × livelli) Numero di celle ancora disponibili	Inizializzarsi con tutte le celle Restituire la Cella richiesta Segnalare se il Tabellone è esaurito
Concorrente	Nome Punteggio corrente Buzzer (stato: attivo/disattivo)	Aggiornare il punteggio (aggiunta o sottrazione) Attivare/disattivare il buzzer Scommettere punti nel Daily Double
Conduttore	Riferimento alla Cella corrente Stato del round	Presentare la risposta di una Cella Validare la domanda di un Concorrente Aggiornare il Tabellone dopo ogni risposta
GameEngine	Tabellone Lista di Concorrenti Conduttore Stato del turno e del round	Avviare un nuovo round Gestire il turno di selezione Eseguire il Final Jeopardy Determinare il vincitore Visualizzare le informazioni
IOTestuale	Campo Tastiera LettoreInput	Mostrare il tabellone e i punteggi a schermo Chiedere nome, scommessa e risposta all'utente Mostrare i messaggi di esito

Relazioni tra le Classi

Due tipi di relazione:

- **Possiede**: l'oggetto B (con chi) fa parte di A (chi) e viene distrutto insieme a lui.
- **Uso**: descrizione dell'Atto d'Uso di come A (chi) fa uso di B (con chi)

Chi	Relazione	Con chi	Osservazioni
GameEngine	Possiede e Usa	Tabellone	GameEngine possiede un solo tabellone per tutta la partita e lo usa per recuperare la Cella scelta e aggiornarne lo stato.
GameEngine	Possiede e Usa	Concorrente	GameEngine possiede i concorrenti e li usa per aggiornarne il punteggio o disattivarne il buzzer.
GameEngine	Possiede e Usa	Conduttore	GameEngine possiede il conduttore e lo usa per presentare le risposte e validare le domande.
Conduttore	Usa	Cella	Il conduttore legge il testo della Cella corrente e chiede a GameEngine di aggiornarla.
Concorrente	Possiede e Usa	Buzzer	Il Concorrente possiede il Buzzer per segnalare la propria risposta.
Tabellone	Possiede	Cella (N×M)	Il Tabellone possiede tutte le celle organizzate in una matrice categorie × livelli.
IOTestuale	Possiede e Usa	Campo	IOTestuale possiede un Campo per visualizzare l'output su terminale.
IOTestuale	Possiede e Usa	Tastiera	IOTestuale possiede una Tastiera per leggere i tasti premuti dall'utente.
IOTestuale	Possiede e Usa	LettoreInput	IOTestuale possiede un LettoreInput per fare domande e ricevere risposte.
GameEngine	Usa	IOTestuale	GameEngine usa IOTestuale sia per l'output che per le risposte dell'utente.

Ciclo di un Round (5 passi)

1. Inizio round: GameEngine chiede a IOTestuale il nome dei Concorrenti, inizializza il Tabellone.
2. Selezione Cella: il Concorrente di turno sceglie categoria e livello; GameEngine recupera la Cella dal Tabellone.
3. Presentazione: il conduttore legge la risposta della Cella; IOTestuale la mostra a schermo.
4. Buzzer e risposta: il primo Concorrente ad attivare il Buzzer fornisce la domanda; il conduttore la valida.

5. Esito: GameEngine aggiorna i punteggi, segna la Cella come usata e IOTestuale mostra il messaggio finale.

Scelte di Progetto

Perché tre classi per l'input/output?

Nella versione attuale il gioco funziona sul terminale: Campo usa cout, Tastiera usa cin. Con questa architettura:

- Campo — definisce dove e come si mostra qualcosa (oggi: terminale, domani: finestra SFML).
- Tastiera — definisce come si legge un tasto (oggi: cin, domani: sf::Event di SFML).
- LettoreInput — coordina la visualizzazione di una domanda e la raccolta della risposta. Se utilizza Campo e Tastiera non cambia, indipendentemente dalla tecnologia usata

Quando si aggiungerà la grafica, GameEngine, Cella, Tabellone, Concorrente e Conduttore non cambieranno la parte logica. Si riscriveranno solo Campo, Tastiera ed eventualmente LettoreInput perché sono loro le uniche classi a sapere come funziona l'input e l'output

Perché GameEngine conosce tutto?

GameEngine possiede Tabellone, Concorrenti e Conduttore ed è l'unico a gestire i turni e l'esito. È una scelta consapevole per questa fase del progetto: permette di avere un unico punto di controllo. Quando il progetto crescerà, si potrà scomporre GameEngine in classi più piccole, ciascuna con una sola responsabilità.

1 Le Classi — Il Progetto

Saper distinguere tra il modello astratto e l'entità concreta

Una classe è lo stampo, non l'oggetto. Cella è la classe; la cella «Scienza per 400» che appare sul tabellone è l'oggetto (l'istanza). Quando progetti, devi sempre chiederti: «Sto descrivendo una categoria di cose, o una cosa specifica?»

Domanda 1 [Classe]

Nel gioco di Jeopardy! uno studente propone di creare una classe chiamata CellaScienzaFacile e una chiamata CellaStoriaDifficile, invece di una singola classe Cella. Dal punto di vista della progettazione OOP, questa scelta è corretta? Motiva la risposta spiegando cosa si perde e cosa si potrebbe fare invece.

 **Suggerimento:** *Pensa a quante classi dovresti creare per coprire tutte le categorie e tutti i livelli. Cosa cambia tra una Cella di Scienza da 200 e una di Storia da 200?*

Domanda 2 [Classe]

Nel modello qui sopra non compare una classe chiamata Round o Turno. Secondo te andrebbe aggiunta? Descrivi cosa rappresenterebbe (cosa 'sa' e cosa 'fa') e se la sua assenza è un problema per il funzionamento del gioco.

 **Suggerimento:** *Prova a elencare le azioni che avvengono in un round. Queste azioni appartengono già a qualche classe esistente o non hanno un 'proprietario'?*

Domanda 3 [Classe]

Un tuo compagno sostiene che Conduttore e Concorrente sono la stessa cosa e che basta una sola classe Partecipante. Ha ragione? Elenca almeno due differenze di comportamento o di stato che giustificano due classi distinte.

 **Suggerimento:** *Guarda il modello: Conduttore ha attributi che Concorrente non ha. E i metodi? Ci sono azioni che può fare solo il conduttore?*

Domanda 4 [Classe]

Perché Cella e Tabellone sono classi diverse?

 **Suggerimento:** *Il Tabellone è l'insieme di tutte le celle organizzate in categorie e livelli: ne esiste uno solo. I metodi sono diversi?*

2 Gli Attributi — Lo Stato

Saper identificare quali dati definiscono l'identità di un oggetto

Gli attributi definiscono cosa un oggetto 'sa' di sé stesso. La domanda giusta non è 'dove posso mettere questo dato?' ma 'a chi appartiene questa informazione per natura?'

Domanda 1 [Attributo]

Se la Cella deve tenere nascosta la domanda corretta fino a quando il Concorrente non risponde, quale attributo booleano aggiungeresti e a quale classe lo assegneresti per garantire che la risposta rimanga segreta? Scrivi la dichiarazione C++ dell'attributo e motiva la scelta della classe.

 **Suggerimento:** Il fatto che una cella sia 'rivelata' o 'nascosta' è una proprietà della Cella stessa o della situazione del Tabellone? Ci possono essere altri contesti in cui una Cella è nascosta?

Domanda 2 [Attributo]

La classe Tabellone contiene una matrice di Cella e un intero celleDisponibili. Perché è necessario mantenere entrambi? Non basterebbe scorrere la matrice ogni volta per contare le celle ancora disponibili? Motiva la risposta considerando anche la leggibilità e l'efficienza del codice.

 **Suggerimento:** Cosa succede se devi chiamare isEsaurito() decine di volte per round? Cosa cambia tra 'ricalcolare' e 'ricordare'?

Domanda 3 [Attributo]

Uno studente aggiunge un attributo punteggioTotale alla classe Concorrente, da aggiornare ogni volta che risponde. Un altro studente dice che è meglio calcolarlo al momento sommando i valori delle celle risposte correttamente. Quale delle due scelte ti sembra più corretta per Jeopardy!, dove il punteggio può anche diminuire? Argomenta.

 **Suggerimento:** In Jeopardy! il punteggio varia in positivo e in negativo. Se salvi il punteggio come attributo, quando lo aggiorni? E nel Daily Double?

Domanda 4 [Attributo]

Perché nella classe Cella la categoria è dichiarata come string e non come int (ad esempio 0=Scienza, 1=Storia, 2=Arte)? Ci sono vantaggi e svantaggi in entrambe le scelte? Descrivi almeno uno per parte.

 **Suggerimento:** Cosa succede quando devi stampare la categoria a schermo? E se devi confrontare due categorie?

3 I Metodi — Il Comportamento

Valutare la capacità di ripartire le azioni in blocchi logici

Un metodo è una responsabilità ben definita. Deve fare una cosa sola, farla bene, e il suo nome deve dire esattamente cosa fa. Se per spiegare un metodo usi la parola 'e', probabilmente dovrebbe essere diviso in due.

Domanda 1 [Metodo]

Descrivi la dichiarazione e il comportamento del metodo `getCella()` della classe `Tabellone`. Cosa deve restituire? Come deve cambiare lo stato interno del `Tabellone` dopo la chiamata? Cosa dovrebbe fare il metodo se la `Cella` richiesta è già stata usata?

 **Suggerimento:** *Un tabellone fisico segna le celle già scelte. Come tieni traccia di quali celle sono ancora disponibili? L'attributo `celleDisponibili` ti dà un indizio.*

Domanda 2 [Metodo]

Il metodo `verificaRisposta()` nella classe `Cella` deve gestire il confronto tra la stringa fornita dal `Concorrente` e la domanda corretta memorizzata. Descrivi a parole la logica che useresti: come gestiresti differenze di maiuscole/minuscole o spazi superflui? Potresti avere più risposte valide per la stessa `Cella`?

 **Suggerimento:** *Inizia normalizzando entrambe le stringhe (tutto minuscolo, trim degli spazi). Poi confrontale. Come gestiresti sinonimi o risposte parzialmente corrette?*

Domanda 3 [Metodo]

Un metodo chiamato `gestisciTurnoConcorrente()` nella classe `Concorrente` fa queste cose: mostra il tabellone, chiede all'utente quale cella scegliere, recupera la `Cella` dal `Tabellone`, mostra la risposta e controlla se la domanda è corretta. È un buon metodo? Identificane i problemi e proponi come suddividerlo.

 **Suggerimento:** *Conta quante 'responsabilità' diverse ha questo metodo. Poi chiediti: quale classe dovrebbe occuparsi di ciascuna di esse?*

Domanda 4 [Metodo]

Il metodo `isEsaurito()` di `Tabellone` deve restituire `true` se non ci sono più celle disponibili. Ha senso che questo metodo controlli internamente l'attributo `celleDisponibili`, oppure dovrebbe ricevere il numero come parametro? Confronta le due versioni.

 **Suggerimento:** *Se `isEsaurito()` accede a `celleDisponibili` internamente, cosa cambia per chi usa la classe `Tabellone`? Chi deve 'sapere' lo stato interno del `Tabellone`?*

4 L'Incapsulamento — Protezione e Interfaccia

Verificare se si sa proteggere l'integrità dei dati

Incapsulare non significa solo mettere 'private' davanti a tutto. Significa decidere consapevolmente quale parte del tuo oggetto è visibile all'esterno (l'interfaccia pubblica) e quale è un dettaglio interno che nessun altro deve toccare direttamente.

Domanda 1 [Incapsulamento]

Nella classe Cella, la domanda corretta è private. Un tuo compagno dice: 'basterebbe renderla public, così chiunque può accedervi direttamente e risparmiamo i metodi'. Spiega almeno due problemi concreti che nascerebbero nel gioco di Jeopardy! se si seguisse questo consiglio.

 **Suggerimento:** Cosa succederebbe se il codice che gestisce il turno del concorrente potesse leggere la risposta corretta prima di rispondere? E se potesse cambiarla?

Domanda 2 [Incapsulamento]

La classe Tabellone ha il metodo mostra() che stampa le celle disponibili. Perché è meglio che questo metodo stia nella classe Tabellone piuttosto che scorrere la matrice di celle dall'esterno e stamparle una a una nel main?

 **Suggerimento:** Cosa succede se in futuro vuoi cambiare il formato di stampa (ad esempio aggiungere colori o evidenziare le celle Daily Double)? In quanti posti del codice dovresti fare modifiche?

⚠ Domanda 3 trabocchetto [Incapsulamento]

La classe Cella ha domandaCorretta come attributo private. Il metodo rivela() cambia lo stato della cella a 'rivelata'. Ma il punteggio assegnato al Concorrente è calcolato da GameEngine, che non sa se la Cella è stata effettivamente rivelata. C'è un problema di coerenza? Come lo risolveresti?

 **Suggerimento:** Attenzione: lo stato 'rivelata' è un attributo della Cella, ma il punteggio è gestito da GameEngine. Chi dei due dovrebbe 'nascondere' o autorizzare l'accesso al valore della Cella?

5 La Composizione — Relazione «Ha un»

Valutare la capacità di costruire oggetti complessi assemblando pezzi semplici

La composizione è il modo principale in cui gli oggetti collaborano: un oggetto 'contiene' un altro oggetto come attributo. Concorrente HA UN Buzzer. Tabellone HA N×M Cella. Costruire bene la composizione significa evitare oggetti troppo grandi e responsabilità duplicate.

Domanda 1 [Composizione]

Sia Concorrente che Conduttore hanno un riferimento allo stato del gioco corrente. Questo significa che stanno 'duplicando' la logica del round? Oppure è la scelta corretta? Giustifica la tua risposta usando il concetto di composizione.

 **Suggerimento:** *Duplicare codice e duplicare la struttura sono due cose diverse. Concorrente e Conduttore condividono informazioni, ma chi implementa la logica di validazione?*

Domanda 2 [Composizione]

Se decidessi di aggiungere un tipo di Cella speciale («Super Cella» che vale il doppio), modificherei la classe Cella esistente o ne creerei una nuova? Motiva la scelta in base alla flessibilità del codice e alla composizione delle classi.

 **Suggerimento:** *Cosa succede alla classe Tabellone se aggiungi proprietà speciali direttamente in Cella? Tutte le celle del tabellone standard avrebbero senso con quei nuovi attributi?*

⚠ Domanda 3 trabocchetto [Composizione]

La classe Tabellone contiene oggetti Cella in una matrice. Potresti costruire il Tabellone senza usare la composizione, memorizzando solo stringhe con categoria e livello come chiavi e calcolando il valore ad ogni accesso. Quali sono i vantaggi e gli svantaggi di questa alternativa rispetto alla composizione?

 **Suggerimento:** *Pensa alla leggibilità del codice e a cosa succede quando aggiungi funzionalità: con sole stringhe, dove metti il metodo isDisponibile() della Cella?*

6 Le Responsabilità — Chi fa cosa

Capire se si sa evitare la creazione di 'Classi Dio' che fanno tutto

Ogni classe deve avere una sola ragione di esistere e una responsabilità chiara. Se una classe conosce troppo delle altre, o fa cose che non le competono, diventa fragile: ogni modifica al gioco richiede di toccarla. La domanda da farsi sempre è: 'Questa azione appartiene davvero a questa classe?'

Domanda 1 [Responsabilità]

Immagina che uno studente crei una sola classe Jeopardy che gestisce tutto: il tabellone, i concorrenti, i turni, il punteggio, la grafica e le regole. Quali sono i problemi pratici di questa scelta quando il gioco deve essere modificato (es. aggiungere un nuovo tipo di concorrente o cambiare le regole del Final Jeopardy)?

 **Suggerimento:** Prova a pensare a cosa devi toccare nella classe Jeopardy ogni volta che cambia anche solo una piccola regola. E se vuoi riusare il Tabellone in un altro gioco a quiz?

Domanda 2 [Responsabilità]

Il metodo `deveRispondere()` nella classe `Concorrente` controlla se il Buzzer è attivo e se il Concorrente ha ancora punti sufficienti. Questo metodo potrebbe stare anche in una classe chiamata `RegoleDelGioco`. Quale delle due posizioni ti sembra più corretta e perché? Ci sono argomenti validi per entrambe?

 **Suggerimento:** Chi 'possiede' la regola del buzzer? È una caratteristica intrinseca del Concorrente o è una regola esterna che qualcuno gli impone?

⚠ Domanda 3 trabocchetto [Responsabilità]

Nella classe `Tabellone` c'è il metodo `mostra()` che stampa le celle a schermo. È corretto che una classe che rappresenta 'dati di gioco' si occupi anche della stampa su schermo? Proponi un'alternativa e discutine i pro e i contro.

 **Suggerimento:** Cosa succede se vuoi usare le stesse classi in un'interfaccia grafica invece che in un terminale? La logica di gioco e la logica di visualizzazione sono la stessa cosa?

Domanda 4 [Responsabilità]

Scrivi la dichiarazione C++ (solo l'intestazione, non l'implementazione) di una classe `Partita` che abbia la responsabilità di coordinare un round del gioco. Quali attributi deve avere? Quali metodi? Giustifica ogni scelta.

 **Suggerimento:** Una classe `Partita` non deve implementare le regole di `Cella` o `Concorrente`: deve coordinarle. Pensa a cosa 'sa' una partita (quali oggetti contiene) e cosa 'fa' (i passi di un round).

Riepilogo delle Competenze

#	Competenza	Descrizione
1	Classe	Saper distinguere il modello astratto dall'entità concreta
2	Attributo	Identificare i dati che definiscono l'identità di un oggetto
3	Metodo	Ripartire le azioni in blocchi logici con responsabilità chiara
4	Incapsulamento	Proteggere l'integrità dei dati e progettare l'interfaccia
5	Composizione	Costruire oggetti complessi assemblando pezzi semplici
6	Responsabilità	Distribuire bene le responsabilità, evitare le Classi Dio